

halfedge：网格变形模块设计文档

src/deformation.rs

halfedge 库

2026-07-05

目录

1	模块定位	1
2	Laplacian 变形	2
2.1	拉普拉斯坐标	2
2.2	线性系统构造	2
2.3	权重与对称写入	2
2.4	Dirichlet 约束处理	3
2.5	流程图	3
3	ARAP 变形	3
3.1	ARAP 能量	3
3.2	Local-Global 迭代	3
3.3	极分解 (Higham 1986)	4
3.4	RHS 构造与约束处理	4
3.5	迭代流程	5
4	3×3 矩阵工具	5
5	退化守卫	6
6	使用示例	6
7	测试覆盖	7

1 模块定位

deformation.rs 在 geometry.rs / linalg.rs / traversal.rs 之上实现两类经典的网格变形 (Mesh Deformation) 算法，参考 Sorkine 等人的工作：

- **Laplacian 变形** (Laplacian Surface Editing, Sorkine et al. 2004) ——通过保留顶点的拉普拉斯坐标（局部细节）同时约束 handle 顶点，求解稀疏线性系统得到变形后的位置；

- **ARAP 变形** (As-Rigid-As-Possible, Sorkine & Alexa 2007) ——Local-Global 迭代，局部步骤对每个顶点 `cell` 计算最佳旋转（极分解），全局步骤固定旋转求解 Poisson 系统更新位置。

类别	API	语义
约束类型	<code>DeformationConstraint</code>	将 <code>vertex</code> 移动到 <code>target_position</code> 的约束描述
Laplacian	<code>laplacian_deformation(mesh, constraints)</code>	保留拉普拉斯坐标 δ_i 的同时满足 <code>handle</code> 约束；分别对 x, y, z 三轴求解
ARAP	<code>arap_deformation(mesh, constraints, iterations)</code>	Local-Global 迭代；先以 Laplacian 解为初值，每轮重估局部旋转 R_i （极分解）再全局求解；大变形下细节保持更好

2 Laplacian 变形

2.1 拉普拉斯坐标

对顶点 i ，设邻接集合 $N(i)$ ，余切权重 w_{ij} （每条无向边在两端各计一次，代码中 `neighbors` 存储 $w_{ij}/2$ ），则离散拉普拉斯坐标为

$$\delta_i = \left(\sum_{j \in N(i)} w_{ij} \right) p_i - \sum_{j \in N(i)} w_{ij} p_j.$$

δ_i 编码了顶点 i 相对于邻域的局部细节（法向、曲率方向）。变形目标是在满足 `handle` 约束的同时，尽量保留 δ_i 。

2.2 线性系统构造

记 p'_i 为变形后位置。对自由顶点 i ：

$$\sum_j L_{ij} p'_j = \delta_i,$$

其中 L 为余切拉普拉斯矩阵。对约束顶点 i （约束目标 c_i ）：

$$p'_i = c_i.$$

分别对 x, y, z 三个分量独立求解。

2.3 权重与对称写入

`build_neighbors_and_weights` 为每个顶点收集邻居索引和余切权重，权重减半（ $w \leftarrow w_{\text{orig}}/2$ ），因为每条无向边在两端的 `VertexRing` 中各被遍历一次。

`SparseSystem::add(i, j, val)` 对称写入 (i, j) 与 (j, i) ，故：

- 非对角总贡献： $-w$ （ i 端） $-w$ （ j 端） $= -2w = -w_{\text{orig}}$ ✓
- 对角只从 i 端写入一次，故需写 $2 \sum (w/2) = \sum w_{\text{orig}}$ ✓

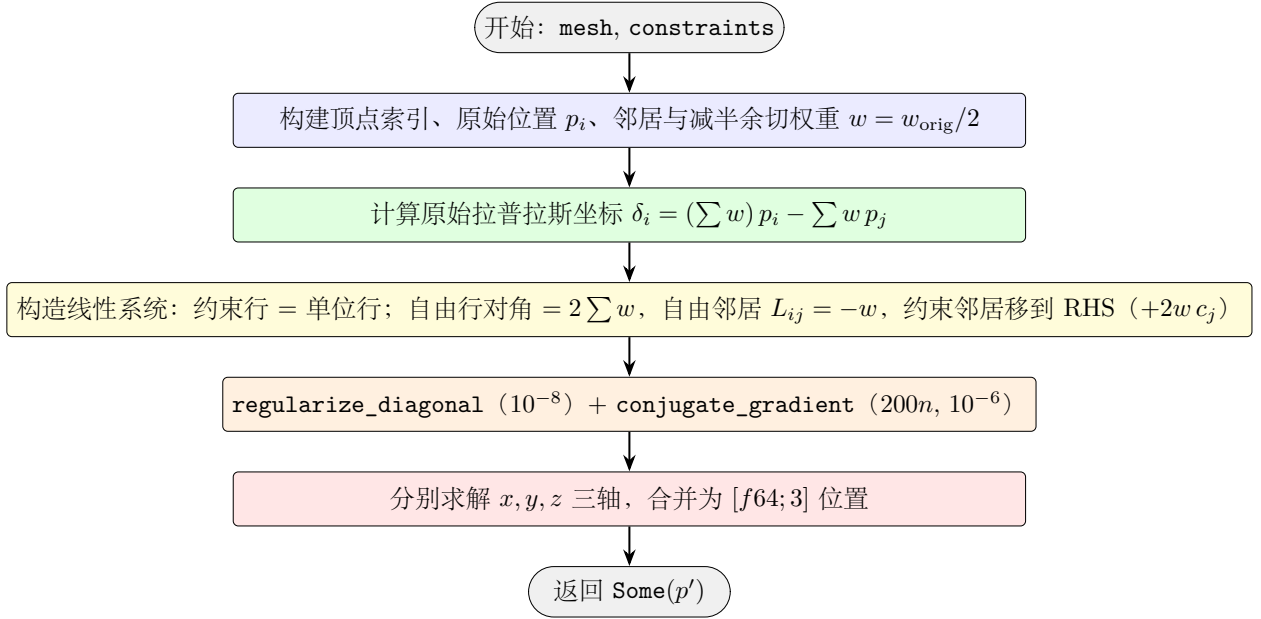
2.4 Dirichlet 约束处理

对自由顶点 i 的邻居 j :

- 若 j 为自由顶点: 写入矩阵 $L_{ij} = -w$ (对称写入后总贡献 $-w_{\text{orig}}$);
- 若 j 为约束顶点: 矩阵中跳过 (i, j) , 将 $-L_{ij} c_j = w_{\text{orig}} c_j = 2w c_j$ 移到 RHS。

矩阵对角 $L_{ii} = \sum_{\text{all } j} w_{\text{orig}}$ (包括约束邻居)。最终系统为 SPD, 调用 `conjugate_gradient` 求解, 迭代上限 $200n$, 相对残差 10^{-6} , 对角正则化 10^{-8} 。

2.5 流程图



3 ARAP 变形

3.1 ARAP 能量

As-Rigid-As-Possible 变形最小化局部刚性偏差:

$$E_{\text{ARAP}} = \sum_i \sum_{j \in N(i)} w_{ij} \|(p'_i - p'_j) - R_i(p_i - p_j)\|^2,$$

其中 R_i 是顶点 i 的 cell (i 及其邻居) 的最佳旋转。 $E = 0$ 当每个 cell 都经历纯刚体变换。

3.2 Local-Global 迭代

局部步骤 (固定 p' , 优化 R): 对每个顶点 i , 构造协方差矩阵

$$S_i = \sum_{j \in N(i)} w_{ij} (p_i - p_j) (p'_i - p'_j)^T,$$

对 S_i 做极分解 $S_i = R_i H_i$ (R_i 正交, H_i 对称半正定), 得到最佳旋转 R_i 。

全局步骤 (固定 R , 优化 p'): 对 E_{ARAP} 关于 p'_i 求导并令零, 得 Poisson 系统

$$\sum_j w_{ij} (p'_i - p'_j) = \frac{1}{2} \sum_j w_{ij} (R_i + R_j) (p_i - p_j),$$

左端即余切拉普拉斯 L (与 Laplacian 变形共用), 右端由当前旋转 R 计算。

3.3 极分解 (Higham 1986)

对 3×3 矩阵 S , 最接近 S 的旋转 R (使 $\|S - R\|_F$ 最小且 R 正交) 由 Newton 迭代求得:

$$R_{k+1} = \frac{1}{2}(R_k + R_k^{-T}), \quad R_0 = S.$$

代码实现 `polar_rotation`, 最多 20 次迭代, 收敛阈值 $\|R_{k+1} - R_k\|_F < 10^{-18}$ 。若 $\det(R) < 0$ (反射), 翻转最后一行修正。

3.4 RHS 构造与约束处理

全局步骤的 RHS 为

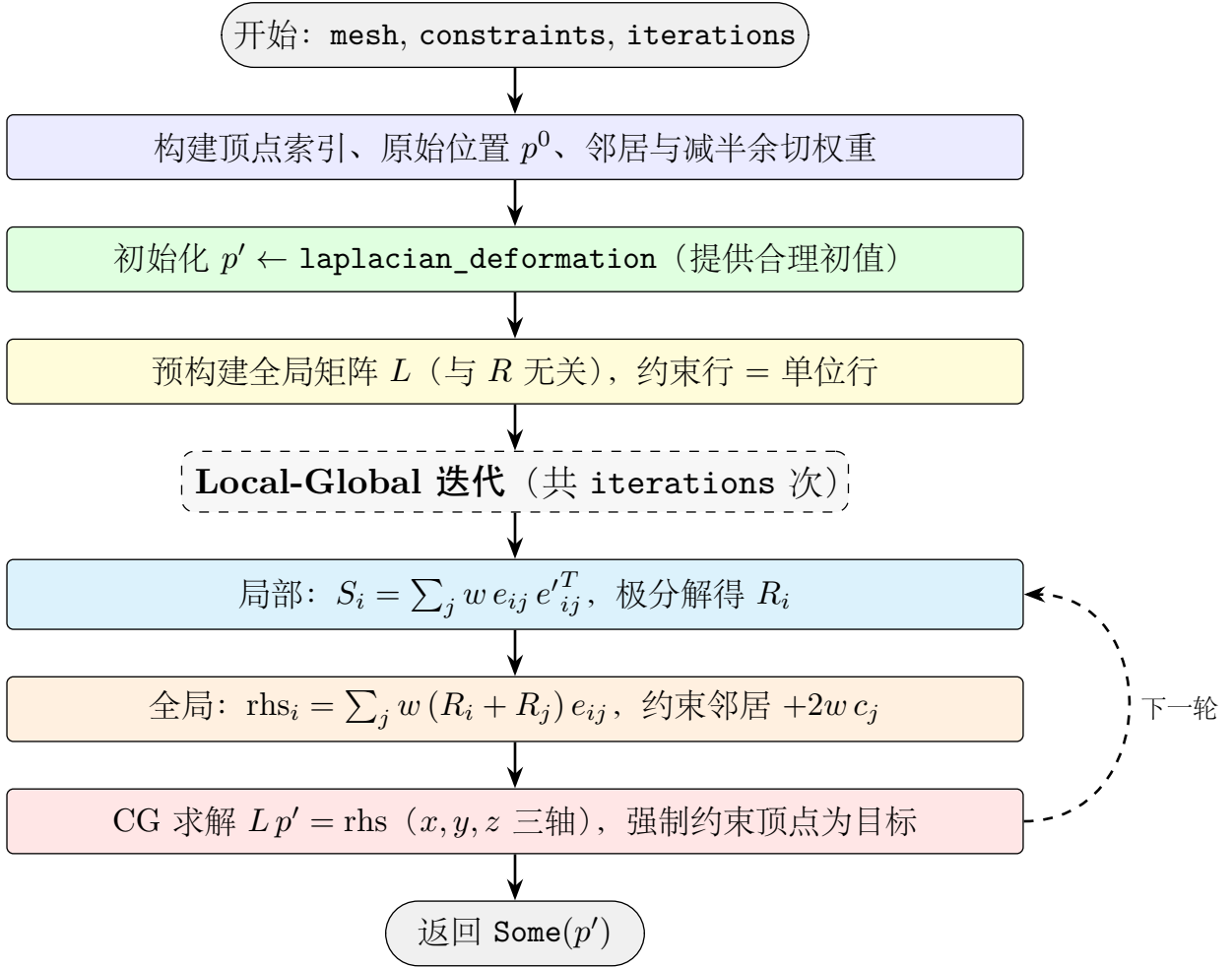
$$\text{rhs}_i = \frac{1}{2} \sum_j w_{\text{orig}} (R_i + R_j) (p_i - p_j).$$

由于 `neighbors` 中 $w = w_{\text{orig}}/2$, 上式等价于

$$\text{rhs}_i = \sum_j w (R_i + R_j) (p_i - p_j).$$

对约束邻居 j , 矩阵中跳过 (i, j) , 将 $-L_{ij} c_j = w_{\text{orig}} c_j = 2w c_j$ 移到 RHS。

3.5 迭代流程



4 3×3 矩阵工具

deformation.rs 内部实现了一套 3×3 矩阵运算 ($\text{Mat3} = [[\text{f64}; 3]; 3]$):

函数	签名	说明
mat3_transpose	$\text{Mat3} \rightarrow \text{Mat3}$	转置
mat3_vec	$(\text{Mat3}, \text{Vec3}) \rightarrow \text{Vec3}$	矩阵 $\times$ 向量
mat3_det	$\text{Mat3} \rightarrow \text{f64}$	行列式
mat3_adjoint	$\text{Mat3} \rightarrow \text{Mat3}$	伴随矩阵 (用于求逆)
mat3_inverse	$\text{Mat3} \rightarrow \text{Option}<\text{Mat3}>$	行列式 $< 10^{-14}$ 返回 None
polar_rotation	$\text{Mat3} \rightarrow \text{Mat3}$	Higham Newton 迭代求最佳旋转

5 退化守卫

函数	退化场景	处理
laplacian_deformation	空网格 ($V = 0$)	返回 <code>None</code>
laplacian_deformation	无约束	返回原始位置 (<code>Some(p)</code>)
laplacian_deformation	对角 $\sum w = 0$	钳为 10^{-10}
laplacian_deformation	CG 未收敛	返回 <code>None</code> (? 传播)
arap_deformation	空网格或无约束	返回 <code>None</code>
arap_deformation	初始化 Laplacian 失败	返回 <code>None</code>
arap_deformation	<code>iterations = 0</code>	至少执行 1 次迭代
polar_rotation	矩阵奇异 (行列式 ≈ 0)	停止迭代, 返回当前 R
polar_rotation	$\det(R) < 0$ (反射)	翻转最后一行修正为旋转
compute_cell_rotations	$\ S\ < 10^{-14}$	退化 cell 使用单位旋转 I

6 使用示例

```

1 use halfedge::deformation::{laplacian_deformation, arap_deformation, DeformationConstraint};
2 use halfedge::storage::{MeshStorage, Vertex};
3 use halfedge::topology_ops::add_triangle;
4
5 //      3x3
6 let mut mesh = MeshStorage::new();
7 let mut vs = Vec::new();
8 for y in 0..3 {
9     for x in 0..3 {
10         vs.push(mesh.add_vertex(Vertex::new([x as f64, y as f64, 0.0])));
11     }
12 }
13 //      ...
14
15 let constraints = vec![
16     DeformationConstraint { vertex: vs[0], target_position: [0.0, 0.0, 0.0] },
17     DeformationConstraint { vertex: vs[8], target_position: [2.0, 2.0, 1.0] },
18 ];
19
20 // Laplacian
21 let deformed = laplacian_deformation(&mesh, &constraints).unwrap();
22
23 // ARAP      5-10
24 let deformed_arap = arap_deformation(&mesh, &constraints, 5).unwrap();

```

7 测试覆盖

测试名	验证点
<code>test_laplacian_deformation_no_constraints_returns_origin</code>	无约束时返回的位置等于原始位置 (容差 10^{-3})
<code>test_laplacian_deformation_single_handle</code>	抬起 $z = 1$; handle 在目标位置; 至少一个邻居有 z 位移
<code>test_laplacian_deformation_two_handles</code>	双 handle (固定 v_0 、抬起 v_8); 两 handle 均在目标位置
<code>test_arap_deformation_basic</code>	4 角点约束 (3 角点固定、1 角点抬起 $z = 1$); 中心顶点 $z > 0$
<code>test_arap_deformation_empty_returns_none</code>	空网格返回 None
<code>test_arap_deformation_no_constraints_returns_none</code>	无约束返回 None
<code>test_polar_rotation_identity</code>	单位矩阵的极分解 $= I$
<code>test_polar_rotation_90deg_around_z</code>	绕 z 轴 90° 旋转矩阵的极分解等于自身
<code>test_polar_rotation_stretch_matrix</code>	含拉伸的矩阵极分解为纯旋转 (行列式 $= 1$ 、正交)
<code>test_laplacian_deformation_icosphere</code>	icosphere 上 Laplacian 变形: handle z 接近目标; 非 handle 顶点有位移

`src/deformation.rs` 共 10 个单元测试, 全库共 394 个。